

Язык программирования Go

Введение и основные конструкции

История создания

- 21 сентября 2007 года – начало проектирования (Кен Томпсон, Роб Пайк, Роберт Гризмер)
- Январь 2008 – начало разработки прототипа компилятора на C (Кен Томпсон)
- Май 2008 – начало разработки front-end для gcc (Ян Тейлор)
- Конец 2008 – к проекту присоединился Расс Кокс
- 11 ноября 2009 – первый релиз

Кен Томпсон, Роб Пайк, Роберт Гризмер, Ян Тейлор, Расс Кокс

Ключевые особенности (1)

- ❶ Простая система типов (отсутствие наследования, статический duck-typing).
- ❷ Поддержка замыканий (функции - полноправные данные).
- ❸ Поддержка параллельно выполняемых сопрограмм (горутин) и каналов.

Ключевые особенности (2)

- 4 Высокая скорость компиляции (пакеты, простая система типов).
- 5 Автоматическое управление памятью (сборка мусора).
- 6 Безопасность (отсутствие арифметики указателей, проверка границ).
- 7 Непротиворечивая грамматика.
- 8 Стандартизованное оформление кода (утилита `gofmt`).

Пример: Hello, World!

```
package main
import "fmt"
func main() {
    fmt.Printf("Hello, World!\n")
}
```

Компиляция и запуск:

```
go build hello.go
./hello
```

Что почитать / порешать

`(https://go.dev/tour/list`

`https://go.dev/doc/effective\_go,`
также `https://github.com/Ox0FACED/effective-go-ru/blob/main/effective-go-ru.pdf`

`https://github.com/slon/shad-go`

`(https://github.com/go101/go101`

- Текст в Unicode. Регистр букв различается.
- Комментарии: `/* ... */` и `// ...`.
- Целочисленные константы и константы с плавающей точкой – как в C.
- Комплексные константы (например, `0.25i`).
- Символьные константы поддерживают Unicode через последовательности `\uxxxx` и `\Uxxxxxxxx`.

25 КЛЮЧЕВЫХ СЛОВ

break	default	func	interface	select
case	defer	go	map	struct
chan	else	goto	package	switch
const	fallthrough	if	range	type
continue	for	import	return	var

Predeclared identifiers

Константы:

`true false iota nil`

Типы:

`any comparable int int8 int16 int32 int64 uint uint8 uint16
uint32 uint64 uintptr float32 float64 complex128 complex64
bool byte rune string error`

Функции:

`min max make len cap new append clear copy close
delete complex real imag panic recover`

Идентификаторы можно переопределять:

```
var true = false
```

Exported identifiers

Регистр первой буквы определяет видимость имени за пределами пакета.

```
package example

const C0 = 9.8
const c1 = 15

var V0 string
var v1 string

func F0() {}
func f1() {}

type T0 int
type t1 string
```

Пакеты всегда называют в нижнем регистре и без подчеркиваний: `fmt`, `grpcmiddleware`

Интерпретируемые (в двойных кавычках):

```
"int main()\n" +  
"{\n" +  
"    printf(\"Hi!\\n\");\n" +  
"    return 0;\n" +  
"}\n"
```

Неинтерпретируемые (в обратных кавычках):

`

```
int main()  
{  
    printf ("Hi!\n");  
    return 0;  
}
```

`

Компилятор вставляет ; после непустой строки, если в конце стоит:

- ❶ идентификатор
- ❷ константа
- ❸ ключевое слово `break`, `continue`, `fallthrough` или `return`
- ❹ спецсимвол `++`, `--`, `)`, `]` или `}`

Пример: автоматическая вставка ;

```
package main
import "fmt"
const (
    ALPHA = iota
    BETA
    GAMMA
)
func main() {
    fmt.Printf("%d\n",
        BETA,
    )
}
```

```
package main;
import "fmt";
const (
    ALPHA = iota;
    BETA;
    GAMMA;
);
func main() {
    fmt.Printf("%d\n",
        BETA,
    );
};
```

4 типа объявлений: `var`, `const`, `type`, `func`

Пакет состоит из множества `.go` файлов.

Порядок объявлений в коде не важен.

Общая форма:

```
var name type = expression
```

Примеры:

```
var i, j, k int           // int, int, int
var b, f, s = true, 2.3, "four" // bool, float64, string
var f, err = os.Open(name) // os.Open returns a file and an error
```

short variable declaration

```
i, j := 0, 1
```

Существующим переменным присваиваются новые значения:

```
in, err := os.Open(infile)
// ...
out, err := os.Create(outfile)
```

Но должна объявляться хотя бы одна новая переменная:

```
f, err := os.Open(infile)
// ...
f, err := os.Create(outfile) // compile error: no new variables
```


Объявление переменных

Полная форма:

```
var x float32
var p, q, r *int
var a, b, c int = 1, 2, 3
```

Блок var:

```
var (
    x float32
    p, q, r *int
    a, b, c int = 1, 2, 3
)
```

Вывод типа:

```
var x = 0.1  
var a, b, c = 'Q', 2, 3.5
```

Сокращённая форма внутри функций (:=):

```
x := 0.1  
a, b, c := 'Q', 2, 3.5
```

Переменная, объявленная, но не инициализированная значением, получает "значение по умолчанию" — нулевое

- Числовые типы (и строковые литералы) — буквально ноль
- Строковый тип — пустая строка
- Указатели, структуры, функции... — `nil` (у каждого типа своё `nil`-значение)

Zero values — значения по умолчанию

Тип	Zero value
bool	false
int, int8...	0
uint, uint8...	0
float32, float64	0.0
complex64, complex128	0+0i
string	""
pointer (*T)	nil

Тип	Zero value
function	nil
interface	nil
slice	nil
map	nil
channel	nil
struct	все поля — zero values
array	все элементы — zero values

```
var (  
    a int           // 0  
    b string        // ""  
    c bool          // false  
    d *float64      // nil  
    e []int         // nil  
    f map[string]int // nil  
    g struct{ x int } // {0}  
)
```

Пустой идентификатор (`_`)

Функция может возвращать несколько значений.

```
a, b, c := f()    // Получаем все три значения  
a, _, c := f()    // Игнорируем второе
```

Пустой идентификатор не создает новую переменную.

Именованные константы

Объявление:

```
const PI float32 = 3.14  
const A, B int = 10, 20  
const x complex64 = 0.25i
```

Блок const:

```
const (  
    PI float32 = 3.14  
    A, B int = 10, 20  
    x complex64 = 0.25i  
)
```

Генератор констант `iota`

`iota` сбрасывается в 0 в начале каждого блока `const` и увеличивается на 1 для каждой следующей константы.

```
const (  
    a = iota    // a == 0  
    b = iota    // b == 1  
    c = iota    // c == 2  
)  
  
const (  
    x = 1 << iota // x == 1  
    y = 1 << iota // y == 2  
    z = 1 << iota // z == 4  
)
```

Опускание повторяющегося выражения

В блоке `const` можно опускать повторяющиеся выражения.

```
const (  
    A int16 = 'a' + iota  
    B  
    C  
)  
// A = 'a', B = 'b', C = 'c'
```


В Go используются **операторы** присваивания, инкремента и декремента.

- **Простое:** `a = b`
- **Составное:** `a += b`
- **Кортежное:** `a, b, c = f(x)`
- **Параллельное:** `a, b = b, a`
- **Инкремент/Декремент:** `a++, a--`

Оператор if

```
if простой_оператор; условие {  
    блок  
} else {  
    блок  
}
```

Пример:

```
if a, b := f(x); a < b {  
    return 0  
} else {  
    fmt.Printf("Hi")  
}
```

- Скобки вокруг условия не ставятся.
- Тело обязательно в {}.

Switch в Golang — основы

```
switch day {  
default: // выполняется последним  
    fmt.Println("Выходной")  
case "пн":  
    fmt.Println("Понедельник")  
case "вт", "ср", "чт":  
    fmt.Println("Середина недели")  
case "пт":  
    fmt.Println("Пятница")  
}
```

- **Нет** автоматического проваливания (fall through)
- Можно перечислять несколько значений через запятую
- default — необязателен
- Выполняется только первый подходящий case

```
switch char {  
case ' ', '\t', '\n', '\r':  
    fmt.Println("Пробельный символ")  
case '0', '1', '2', '3', '4', '5', '6', '7', '8', '9':  
    fmt.Println("Цифра")  
default:  
    fmt.Println("Другой символ")  
}
```

Switch без выражения — switch true

```
x := 42

if x < 0 {
    fmt.Println("Отрицательное")
} else if x == 0 {
    fmt.Println("Ноль")
} else if x < 10 {
    fmt.Println("Маленькое")
} else if x < 100 {
    fmt.Println("Среднее")
} else {
    fmt.Println("Большое")
}
```



```
x := 42

switch {
case x < 0:
    fmt.Println("Отрицательное")
case x == 0:
    fmt.Println("Ноль")
case x < 10:
    fmt.Println("Маленькое")
case x < 100:
    fmt.Println("Среднее")
default:
    fmt.Println("Большое")
}
```

```
switch x { ... }    // сравнивает x с case-выражениями
switch { ... }      // то же самое, что switch true { ... }
```

Switch с инициализацией

```
switch x := getValue(); {  
case x < 0:  
    fmt.Printf("x = %d - отрицательное\n", x)  
case x == 0:  
    fmt.Printf("x = %d - ноль\n", x)  
default:  
    fmt.Printf("x = %d - положительное\n", x)  
}  
// x здесь уже не видна
```

```
switch n := getNumber(); n % 2 {  
case 0:  
    fmt.Println(n, "чётное")  
case 1:  
    fmt.Println(n, "нечётное")  
}
```

```
switch file, err := os.Open(fname);  
↪ {  
case err != nil:  
    log.Fatal(err)  
default:  
    defer file.Close()  
    // работа с file  
}
```

fallthrough — принудительное проваливание

```
switch num {  
case 1:  
    fmt.Println("Один")  
    fallthrough  
case 2:  
    fmt.Println("Два")  
    fallthrough  
case 3:  
    fmt.Println("Три")  
default:  
    fmt.Println("Что-то ещё")  
}
```

Вывод для num = 1:

Один
Два
Три

Вывод для num = 2:

Два
Три

- fallthrough должен быть последним оператором в case
- Он безусловно передаёт управление в следующий case
- Используется редко

Type switch — переключение по типу

```
func describe(i interface{}) {  
    switch v := i.(type) {  
    case int:  
        fmt.Printf("Целое число: %d\n", v)  
    case string:  
        fmt.Printf("Строка: %q, длина %d\n", v, len(v))  
    case bool:  
        fmt.Printf("Булево: %v\n", v)  
    case []int:  
        fmt.Printf("Срез int'ов: %v\n", v)  
    case nil:  
        fmt.Println("Это nil")  
    default:  
        fmt.Printf("Неизвестный тип: %T\n", v)  
    }  
}
```

describe(42)	Целое число: 42
describe("hello")	Строка: "hello", длина 5
describe(true)	Булево: true
describe(nil)	Это nil

Сравнение if-else и switch

```
if err := process(); err != nil {  
    return err  
}
```

```
switch {  
case score >= 90:  
    grade = "A"  
case score >= 80:  
    grade = "B"  
default:  
    grade = "F"  
}
```


Классический пример

```
func f() bool {  
    return false  
}  
  
func isTrue() bool {  
    switch f() {  
    case true:  
        return true  
    default:  
        return false  
    }  
}
```

Результат: false

```
func f() bool {  
    return false  
}  
  
func isTrue() bool {  
    switch f() // компилятор вставит ;  
    { // и прочитает как switch f();  
    ↪ true  
    case true:  
        return true  
    default:  
        return false  
    }  
}
```

Результат: true

Цикл for

Полная форма (как for C):

```
for простой_оператор; условие; простой_оператор {  
    блок  
}
```

Форма "while":

```
for условие {  
    блок  
}
```

Бесконечный цикл:

```
for {  
    блок  
}
```

break и continue работают как обычно.

Примеры циклов

Числа Фибоначчи:

```
for a, b := 1, 1; b < 100; a, b = b, a+b {  
    fmt.Println("%d\n", b)  
}
```

Быстрое возведение в степень:

```
res := 1  
for n > 0 {  
    if n & 1 == 1 {  
        res *= x  
    }  
    x *= x  
    n >>= 1  
}
```

Примитивные типы

Тип	Знак	Бит	Мин	Макс
int8	да	8	-128	127
uint8 (byte)	нет	8	0	255
int16	да	16	-32768	32767
uint16	нет	16	0	65535
int32 (rune)	да	32	-2^{31}	$2^{31} - 1$
uint32	нет	32	0	$2^{32} - 1$
int64	да	64	-2^{63}	$2^{63} - 1$
uint64	нет	64	0	$2^{64} - 1$
int	да	32/64	—	—
uint	нет	32/64	—	—
uintptr	нет	"дикий" указатель		
float32	да	32	$\pm 3.4 \cdot 10^{\pm 38}$ (7 цифр)	
float64	да	64	$\pm 1.7 \cdot 10^{\pm 308}$ (15 цифр)	
complex64	(float32, float32)			
complex128	(float64, float64)			

```
var A [10]int
const N = 100
var B, C, D [N]float64
var M [5][4]int    // эквивалентно [5]([4]int)
```

- Индексация: `A[i]`
- Выход за границы запрещен.
- Длина: `len(A)`

range для перебора массива

```
package main

import "fmt"

func main() {
    var A [10]int
    for i := 0; i < 10; i++ {
        fmt.Scanf("%d", &A[i])
    }
    for i, x := range A {
        fmt.Printf("A[%d] = %d\n", i, x)
    }
}
```

Создание и инициализация безымянного массива:

```
[10]int{1, 2, 3}           // остальные - нули  
[...]int{1, 2, 3}         // массив размера 3
```

Пример передачи массива

```
package main

import "fmt"

func f(a [5]int) {
    for i, x := range a {
        fmt.Printf("%d) %d; ", i, x)
    }
}

func main() {
    f([...]int{10, 20, 30, 40, 50})
}
```

Вывод: 0) 10; 1) 20; 2) 30; 3) 40; 4) 50;


```
var p *int
var q **int
var ap *[10]int
```

- Разыменование: *p
- Взятие адреса: &(A[i])
- Арифметика указателей **запрещена**.
- Индексация указателей **запрещена**.
- void*-указателей **нет**.

Массивы в Go используются редко из-за фиксированного размера.

Срез — это структура данных, описывающая фрагмент массива:

- указатель на первый элемент фрагмента;
- длина (количество элементов);
- ёмкость (capacity).

```
var A []int
var P []*int
var M [][]float64
```

- К срезам применима операция индексации `[]`.
- Выход за границы **запрещен**.

Срезы (slices) - окно в подлежащий массив

len=4, cap=8
slice := array[2:6]



Подлежащий массив (capacity = 10)
[10]int{0,1,2,3,4,5,6,7,8,9}

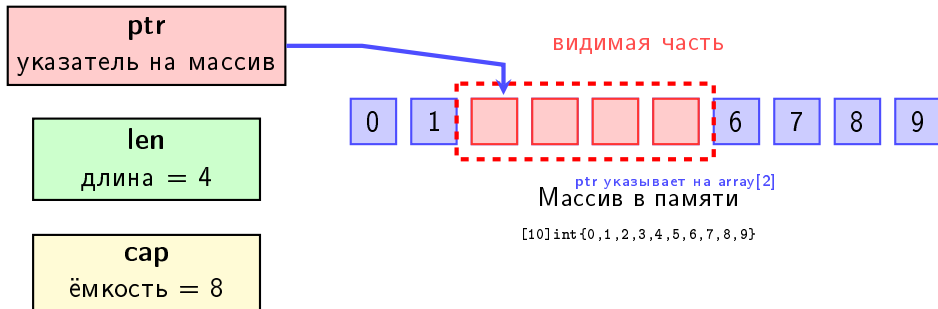
len=5, cap=6
slice2 := slice[2:7]



```
array :=  
↪ [10]int{0,1,2,3,4,5,6,7,8,9}  
slice := array[2:6]  
// len=4, cap=8
```

```
slice2 := slice[2:7]  
// len=5, cap=6  
  
slice3 := array[:8]  
// len=8, cap=10
```

Внутреннее устройство среза

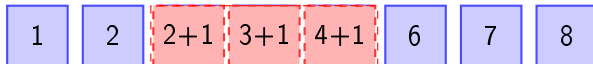


```
struct { ptr *array len int cap int }
```

- **ptr** указывает на первый элемент среза (не на начало массива!)
- **len** — количество видимых элементов
- **cap** = len(массива) - (ptr - начало массива)
- Несколько срезов могут ссылаться на один массив

Изменение массива через срез

`slice := array[2:5]`



После `slice[1] = 99`:



- Срез не копирует данные — он показывает окно в массив
- Изменение через срез меняет исходный массив
- Другие срезы, показывающие ту же область, тоже увидят изменения

```
array := [8]int{1,2,3,4,5,6,7,8}
slice := array[2:5] // [3,4,5]

slice[1] = 99      // меняем элемент среза

fmt.Println(array) // [1,2,3,99,5,6,7,8] - массив изменился!
fmt.Println(slice) // [3,99,5] - срез показывает новое значение
```

append и переполнение ёмкости

1. Исходный срез (cap=4):

`s := []int{10,20,30} (len=3, cap=4)`



2. Append (в пределах cap):

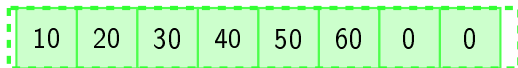
`s = append(s, 40) (len=4, cap=4)`



3. Append (превышение cap) — создаётся новый массив:



`s = append(s, 50, 60) (len=6, cap=8)`



```
s := make([]int, 3, 4) // len=3, cap=4
s[0], s[1], s[2] = 10, 20, 30

s = append(s, 40)      // len=4, cap=4 (ещё тот же массив)
s = append(s, 50, 60)  // cap превышена - новый массив (len=6, cap=8)
```


len и cap для срезов

- len — количество "занятых" элементов.
- cap — общая вместимость.

```
package main

import "fmt"

func main() {
    var a []int
    fmt.Printf("%d %d\n", len(a), cap(a))
}
```

Вывод: 0 0

Составной литерал среза

```
package main

import "fmt"

func main() {
    a := []int{1, 2, 3}
    fmt.Printf("%d %d\n", len(a), cap(a))
}
```

Вывод: 3 3

Операция среза

```
package main
import "fmt"
func main() {
    array := [7]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
    slice := array[2:5] // c, d, e
    for _, x := range slice {
        fmt.Printf("%c ", x)
    }
}
```

Вывод: c d e

Значения по умолчанию в срезах

```
array := [7]byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
```

```
array[:4] []byte{'a', 'b', 'c', 'd'}
```

```
array[2:] []byte{'c', 'd', 'e', 'f', 'g'}
```

```
array[:] []byte{'a', 'b', 'c', 'd', 'e', 'f', 'g'}
```

Срез – это окно в массив

Изменение среза меняет исходный массив.

```
package main
import "fmt"
func main() {
    A := [5]int{1, 2, 3, 4, 5}
    b := A[2:] // b ссылается на A[2], A[3], A[4]
    b[1] = 10  // меняет A[3]
    for _, x := range A {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 1 2 3 10 5

`cap(slice) = len(array) - start`

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]
    fmt.Printf("%d %d", len(slice), cap(slice))
}
```

Вывод: 10 90

Срез от среза

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]    // len=10, cap=90
    slice2 := slice[5:]  // от индекса 5 (всего 5 элементов)
    fmt.Printf("%d %d", len(slice2), cap(slice2))
}
```

Вывод: 5 85 (cap = 90 - 5)

Расширение среза (в пределах ёмкости)

```
package main
import "fmt"
func main() {
    var A [100]int
    slice := A[10:20]    // len=10, cap=90
    slice2 := slice[:30] // увеличиваем длину slice2 до 30
    fmt.Printf("%d %d", len(slice2), cap(slice2))
}
```

Вывод: 30 90

Копирование срезов: copy

```
package main
import "fmt"
func main() {
    A := [8]int{0, 1, 2, 3, 4, 5, 6, 7}
    copy(A[1:3], A[5:]) // копирует A[5], A[6] в A[1], A[2]
    for _, x := range A {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 0 5 6 3 4 5 6 7

Создание срезов: make

```
A := make([]int, length, capacity)
```

Создаёт в куче массив размера `capacity` и возвращает срез длины `length` на него.

```
A := make([]int, 0, 100) // длина 0, ёмкость 100
```

Оператор `make` выделяет память под структуру, а также инициализирует и возвращает её. В отличие от него, оператор `new` выделяет память под нулевое значение типа и возвращает указатель на это значение.

Сокращённая форма make

```
A := make([]int, capacity)
```

Длина и ёмкость будут равны capacity.

```
A := make([]int, 100) // длина 100, ёмкость 100
```

Динамическое расширение: `append`

```
slice = append(slice, x1, x2, ...)
```

Добавляет элементы в конец среза. При нехватке ёмкости выделяет новый массив.

Пример append

```
package main

import "fmt"

func main() {
    s := make([]int, 0, 5)
    s = append(s, 1)
    for a, b := 1, 1; b < 100; a, b = b, a+b {
        s = append(s, b)
    }
    for _, x := range s {
        fmt.Printf("%d ", x)
    }
}
```

Вывод: 1 1 2 3 5 8 13 21 34 55 89

- Встроенный тип `string`.
- Неизменяемые массивы `byte` (UTF-8).
- Индексация: доступ к **байтам** (`s[i]`). Запись запрещена.
- Вырезание: `s[2:5]` возвращает **строку** (байты).
- Конкатенация: `s + t` (создаёт новую строку).

`len` возвращает количество **байт**.

```
package main
import "fmt"
func main() {
    s := "кошка"
    fmt.Printf("len(%q) == %d\n", s, len(s))
}
```

Вывод: `len("кошка") == 10`

Операция: $T(x)$

- `string(1072) → "a"` (кодировка точка)
- `string([]byte{...}) → строка из байт`
- `[]byte(s) → срез байт (копия)`
- `string([]rune{...}) → строка из рун`
- `[]rune(s) → срез рун (копия)`

Пример: строка в срез рун

```
package main
import "fmt"
func main() {
    s := "кошка"
    a := ([]rune)(s) // или []rune(s)
    for _, x := range a {
        fmt.Printf("%c - %d\n", x, x)
    }
}
```

к - 1082

о - 1086

ш - 1096

к - 1082

а - 1072

Объявление функций

```
func имя(параметры) результат {  
    блок  
}
```

- Параметры: (p1 тип1, p2 тип2, ...)
- Можно сокращать: (a, b int, c string)
- Возвращаемые значения указываются как параметры или просто тип.

Именованные возвращаемые значения

```
func minmax(a, b int) (min, max int) {  
    if a < b {  
        min, max = a, b  
    } else {  
        max, min = a, b  
    }  
    return // "золый" return возвращает min, max  
}
```

```
func min(a, b int) int {  
    if a < b {  
        return a  
    }  
    return b  
}
```

Неименованные возвращаемые значения

```
func minmax(a, b int) (int, int) {  
    if a < b {  
        return a, b  
    }  
    return b, a  
}
```

Если функция ничего не возвращает:

```
func hello(name string) {  
    fmt.Printf("Hello, %s!\n", name)  
}
```

Композиция функций

```
func g(x int) (int, int) { return x, -x }  
  
func f(a, b int) int { return a + b }  
  
func main() {  
    fmt.Printf("%d", f(g(5))) // f(5, -5) = 0  
}
```

Вывод: 0

Функции с переменным числом аргументов

Вариадик-аргумент — всегда последний в списке:

```
func concat(s ...string) string {
    length := 0
    for _, x := range s { length += len(x) }
    result, pos := make([]byte, length), 0
    for _, x := range s {
        copy(result[pos:], ([]byte)(x))
        pos += len(x)
    }
    return string(result)
}

func main() {
    fmt.Print(concat("alpha", "beta", "gamma"))
}
```

Типичный случай, когда требуется передать в аргументы кортеж неизвестной длины (0 или больше) — `fmt.Printf`, `fmt.Sprintf` (кортеж: переменные любых типов, которым соответствуют модификаторы в первом аргументе, вариадик `...any`).

Передача среза в ...

```
func main() {  
    a := []string{"alpha", "beta", "gamma"}  
    fmt.Print(concat(a...))  
}
```

При использовании методов `fmt` напрямую:

```
a := []any{"alpha", "beta", "gamma"}  
fmt.Println(a) // [alpha beta gamma]  
fmt.Println(a...) // alpha beta gamma
```

Функции как данные

```
var f func(a, b int)
var g func(x float32) string
var A []func([]byte) (int, int)
```

Тип функции: `func` (сигнатура).


```
func main() {  
    var x int  
    f := func(y int) int { return x + y }  
    x = 5  
    fmt.Printf("%d", f(10))  
}
```

Вывод: 15

От стрелочной нотации к синтаксису Go

Стрелочная нотация:

```
int -> string
(int, string) -> bool
(int) -> (string, error)
(int -> bool) -> float64
int -> (string -> []byte)
```

Синтаксис Go:

```
func(int) string
func(int, string) bool
func(int) (string, error)
func(func(int) bool) float64
func(int) func(string) []byte
```

Стираем стрелки — получаем Go! Стрелочная нотация → заменяется на `func(...)` ...

Правоассоциативность функциональных типов

Стрелочная нотация:

`int -> string -> []byte`

`int -> (string -> []byte)`
└───────────┘
 |
 v
 функция

Синтаксис Go:

```
func(int) func(string) []byte
```

`func(int) (func(string) []byte)`
└───────────┘
 |
 v
 функция

```
type StringProcessor func(int) func(string) []byte

// Функция, возвращающая функцию
func makePrefixer(prefix string) func(string) string {
    return func(s string) string {
        return prefix + s
    }
}
```

Функции, возвращающие кортежи (несколько значений)

Стрелочная нотация:

(string, int) ->
(string, error)

```
func(string, int) (string, error)
```

```
func(string, int) (string, error)
      \_____/ \_____/
      параметры возвращаемые
                        значения
```

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division by zero")
    }
    return a / b, nil
}
```

Пример: обобщённая сортировка

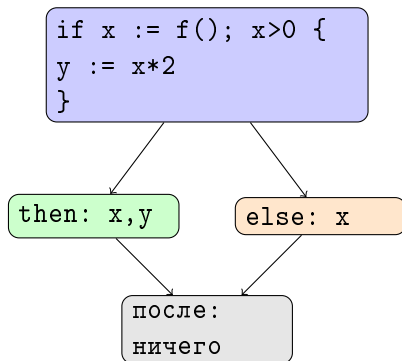
```
func sort(n int, less func(int, int) bool, swap func(int, int)) {  
    for j := n-1; j > 0; j-- {  
        k := j  
        for i := j-1; i >= 0; i-- {  
            if less(k, i) { k = i }  
        }  
        swap(j, k)  
    }  
}  
  
func main() {  
    A := []int{5, 2, 6, 1, 3, 5, 0, 9}  
    sort(len(A),  
        func(i, j int) bool { return A[i] < A[j] },  
        func(i, j int) { A[i], A[j] = A[j], A[i] },  
    )  
    for _, x := range A { fmt.Printf("%d ", x) }  
}
```

Рекурсивные замыкания

```
func main() {  
    var fact func(int) int  
    fact = func(x int) int {  
        if x == 0 {  
            return 1  
        }  
        return x * fact(x-1)  
    }  
    fmt.Printf("%d", fact(4))  
}
```

Scope в if

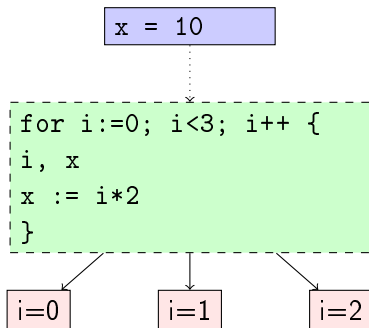
```
func main() {  
    if x := getValue(); x > 0 {  
        y := x * 2  
        fmt.Println(x, y) // x и y  
        ↪ видны  
    } else {  
        fmt.Println(x)    // x  
        ↪ видна  
        // y не видна  
    }  
    // x и y не видны  
}
```



```
if err := process(); err != nil {  
    return err  
}  
// err здесь уже не видна
```

Scope в for

```
func main() {  
    x := 10  
  
    for i := 0; i < 3; i++ {  
        fmt.Println(i, x) // i и x  
        ↪ видны  
        x := i * 2        // новая x  
        ↪ внутри цикла  
        fmt.Println(x)  
    }  
  
    fmt.Println(i) // panic: i не видна  
    fmt.Println(x) // 10 - внешняя x не  
    ↪ изменилась  
}
```



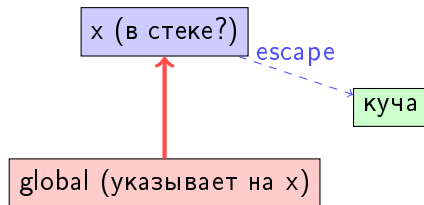
Переменная, объявленная в инициализации цикла, живёт только на время цикла. Каждая итерация получает свою копию (Go ≥ 1.22).

Score и память — время жизни

```
var global *int

func example() {
    x := 42
    global = &x // x уходит в кучу!
} // x всё ещё жива через global
```

```
func example2() {
    y := new(int)
    *y = 100
    // y может остаться на стеке
}
```



если на x ссылаются
после выхода из score,
она перемещается в кучу

Компилятор Go анализирует, выходит ли переменная за пределы своей области видимости. Если да — переменная создаётся в куче, если нет — может остаться на стеке.

Именованные типы и типовые литералы

Типовой литерал — изображение типа в программе.

```
*int []int func(int, int) int
```

Именованные типы — типы, определяемые только именем (включая примитивные).

```
int float64
```

Пользовательские именованные типы

```
type Color int
type Name string
type ColorSlice []Color
type Matrix [3][3]float32
type Filter func([]int) []int
```

Не путаем с алиасами типов:

```
// здесь ColorCode - буквально тот же int
type ColorCode = int
```

Базовый (Underlying) тип

- Если `T` — примитивный или неименованный тип (литерал — указатель, слайс, отображение, функция...), то его базовый тип — `T`.
- Если `T` — пользовательский тип (`type T литерал`), то базовый тип для `T` — базовый тип этого литерала.

(Определение рекурсивное.)

"Неименованный" тип — тот, самый верхний конструктор которого не именован. Например тип `map[MyType]MyOtherType` — не именованный.

Пример базового типа

```
type Color int
type Colour Color
type NiceColour Colour
type ColorSlice []Color
type ColorSet ColorSlice
```

- Базовый тип для Color, Colour, NiceColour — int.
- Базовый тип для ColorSlice и ColorSet — []Color (он неименованный, поскольку представлен не отдельным именем).

x типа Q можно присвоить переменной типа T , если:

- 1 Q идентичен T , то есть представлен тем же именем, либо является неименованным и имеет такой же типовой литерал;
- 2 Q и T имеют идентичные базовые типы, и хотя бы один из них неименованный;
- 3 T — интерфейс, и Q его реализует;
- 4 $x = nil$ и T — указатель, функция, срез, канал, отображение или интерфейс;
- 5 x — примитивная константа, подходящая для базового типа T .

Примеры совместимости

```
type IntSlice []int
type Ints []int
var (
    a []int = []int{1, 2, 3}
    x IntSlice
    y Ints
)
x = a // ок, именованный и безымянный
y = a // ок, то же
a = x // ок, безымянный и именованный
a = y // ок то же
x = y // ошибка, два именованных
```

```
type Color int
var (
    i int = 1
    c Color
)
c = 1 // ок, именованный и примитивная константа
c = i // ошибка, именованный и именованный
i = c // ошибка, то же
```

Рекурсивные типы

В объявлении типа можно использовать имя самого объявляемого типа.

```
package main
import "fmt"
type T *T
func main() {
    var w, x, y, z T
    w, x, y, z = &x, &y, &z, nil
    for i := w; i != nil; i = *i {
        fmt.Printf("* ")
    }
}
```

Значение переменной типа T — указатель на неё. Поэтому цикл напечатает * * *

Отображения (map)

```
var d map[string]int
var x map[int]func(int) int
```

- Неинициализированный map равен nil.
- Ключи должны поддерживать == и != (быть comparable).
- Нельзя использовать функции, срезы, map'ы в качестве ключей (математический смысл — это всё отображения со сложно разрешимым, если вообще разрешимым, равенством).

Запись типа отображения

Логика та же, что при записи типов функций, массивов и срезов. Посмотрим с точки зрения математики.

- $[n]T$ — тип однозначного отображения из непрерывного отрезка натуральных чисел в тип $T: [0, n - 1] \rightarrow T$.
- $[]T$ — тип однозначного отображения из начального отрезка натуральных чисел неизвестной длины в тип $T: [0, n - 1] \rightarrow T$.
- $\text{map}[K]T$ — тип однозначного отображения из типа K в тип $T: K \rightarrow T$. Особенность языка Go — это отображение определено на любом значении из K и возвращает нулевое значение из T .

`range` по слайсам и мапам на каждой итерации возвращает пару (аргумент отображения, результат). Для слайса это пара $(i, s[i])$, для мапы — $(key, m[key])$.

С точки зрения математики:

- тип отображения — $\text{map}[K] \ T$ — тип валидных (чистых, однозначных) *функций* из K в T . Дополнительно требуется, чтобы эта функция возвращала ненулевые значения лишь на конечном множестве элементов из K .
- тип функции — $\text{func}(x \ K) \ T$ — тип недетерминированных, неоднозначных *отображений* из K в T с возможными побочными эффектами.

Nota Bene: факт однозначности отображения, а именно единственности значения для каждого из ключей, частично выражен в требовании к типу ключа (`comparable`, встроенное равенство).

Создание map

```
make(map[K]V, capacity)
make(map[K]V)
```

```
dict := make(map[string]int, 100)
```

`len()` возвращает количество пар.

Напомним: `make(TYPE, ATTRIBUTES)` выделяет память под сущность (слайс, мапу или канал), инициализирует и возвращает эту сущность. Вызов `new(map[K]V)` создаст только указатель на нулевую не инициализированную мапу. Поэтому идиоматично — `make`.

Добавление и поиск в map

```
a[k] = v           // добавление/замена  
v := a[k]         // поиск (если нет ключа --- zero value)
```

Пример map

```
package main
import "fmt"

func main() {
    a := make(map[string]int, 16)
    a["alpha"] = 10
    a["alpha"] = 20
    fmt.Printf("%d %d\n", a["alpha"], a["beta"])
}
```

Вывод: 20 0

Проверка существования ключа

```
v, ok := a[k]
```

```
package main
import "fmt"

func main() {
    a := make(map[string]int)
    a["alpha"] = 10
    v, ok := a["alpha"]
    fmt.Printf("%d %v\n", v, ok)
}
```

Вывод: 10 true

Удаление из map

```
delete(m, k)
```

```
package main
import "fmt"

func main() {
    a := make(map[string]int)
    a["alpha"] = 10
    delete(a, "alpha")
    v, ok := a["alpha"]
    fmt.Printf("%d %v\n", v, ok)
}
```

Вывод: 0 false

Составной литерал map

```
package main
import "fmt"

func main() {
    a := map[int]string{
        10: "alpha", 20: "beta", 30: "gamma",
    }
    fmt.Printf("%s\n", a[20])
}
```

Вывод: beta

```
struct {  
    x, y int  
    name string  
}
```

Именованные структуры

```
type Point3D struct {  
    x, y, z float64  
}  
  
type Node struct {  
    prev, next *Node  
    value string  
}  
  
type List struct {  
    tail List // так нельзя!  
    x int  
}
```

Традиционная функция:

```
func Distance(p, q Point) float64 {  
    return math.Hypot(q.X-p.X, q.Y-p.Y)  
}
```

Метод типа Point:

```
func (p Point) Distance(q Point) float64 {  
    return math.Hypot(q.X-p.X, q.Y-p.Y)  
}
```

- Специальный аргумент называется **receiver**
- В Go не используются `this` или `self`
- Receiver обычно называют по первой букве имени типа

С пользовательским именованным типом (или указателем на него) можно связать методы.

Метод — функция с параметром-приёмником:

```
func (приёмник) имя сигнатура блок
```

Приёмник:

- имя_параметра пользовательский_именованный_тип
- имя_параметра *пользовательский_именованный_тип

```
func TestPoint(t *testing.T) {  
    p := Point{1, 2}  
    q := Point{4, 6}  
    fmt.Println(Distance(p, q))    // "5", function call  
    fmt.Println(p.Distance(q))    // "5", method call  
}
```

Тип-срез с методом:

```
type Path []Point

func (path Path) Distance() float64 {
    sum := 0.0
    for i := range path {
        if i > 0 {
            sum += path[i-1].Distance(path[i])
        }
    }
    return sum
}
```

Path (пример)

```
perim := Path{
    {1, 1},
    {5, 1},
    {5, 4},
    {1, 1},
}
fmt.Println(perim.Distance()) // "12"
```


Pointer Receiver

```
func (p *Point) ScaleBy(factor float64) {  
    p.X *= factor  
    p.Y *= factor  
}
```

Nil receiver

```
type IntList struct {  
    Value int  
    Tail *IntList  
}  
  
func (list *IntList) Sum() int {  
    if list == nil {  
        return 0  
    }  
    return list.Value + list.Tail.Sum()  
}
```

Встраивание (Embedding)

```
type Point struct{ X, Y float64 }  
  
type ColoredPoint struct {  
    Point  
    Color color.RGBA  
}
```

```
red := color.RGBA{255, 0, 0, 255}
blue := color.RGBA{0, 0, 255, 255}

var p = ColoredPoint{Point{1, 1}, red}
var q = ColoredPoint{Point{5, 4}, blue}

fmt.Println(p.Distance(q.Point)) // "5"
p.ScaleBy(2)
q.ScaleBy(2)
fmt.Println(p.Distance(q.Point)) // "10"
```

Встраивание методов через анонимные поля

Если структура S содержит анонимное поле T :

- Методы T входят в набор методов S .

Если поле $*T$:

- Методы $*T$ (и T) входят в набор методов S .

Для $*S$:

- Методы $*T$ (и T) входят в набор методов $*S$.

Встраивание != Наследование

```
var p = ColoredPoint{Point{1, 1}, red}  
var q = ColoredPoint{Point{5, 4}, blue}  
  
p.Distance(q) // compile error: cannot use q (ColoredPoint) as Point
```

Method Values

```
p := Point{1, 2}
q := Point{4, 6}

distanceFromP := p.Distance
fmt.Println(distanceFromP(q))

var origin Point
fmt.Println(distanceFromP(origin))

scaleP := p.ScaleBy
scaleP(2)
scaleP(3)
scaleP(10)
```

Method Expressions

```
p := Point{1, 2}
q := Point{4, 6}

distance := Point.Distance // method expression
fmt.Println(distance(p, q)) // "5"
fmt.Printf("%T\n", distance) // "func(Point, Point) float64"

scale := (*Point).ScaleBy
scale(&p, 2)
fmt.Println(p) // "{2 4}"
fmt.Printf("%T\n", scale) // "func(*Point, float64)"
```


Инкапсуляция — через синтаксис

Делаем поле невидимым для внешних пакетов, а методы — видимыми.

```
type Counter struct { n int }  
  
func (c *Counter) N() int { return c.n }  
func (c *Counter) Increment() { c.n++ }  
func (c *Counter) Reset() { c.n = 0 }
```

Геттеры/сеттеры для простых типов не приветствуются, в этом случае можно сделать поля доступными для внешних пакетов.

```
func Fprintf(w io.Writer, format string, args ...any) (int, error)

func Printf(format string, args ...any) (int, error) {
    return Fprintf(os.Stdout, format, args...)
}

func Sprintf(format string, args ...any) string {
    var buf bytes.Buffer
    Fprintf(&buf, format, args...)
    return buf.String()
}
```

Интерфейс Writer

```
// Writer is the interface that wraps the basic Write method.  
type Writer interface {  
    // Write writes len(p) bytes from p to the underlying data stream.  
    // It returns the number of bytes written from p (0 <= n <= len(p))  
    // and any error encountered that caused the write to stop early.  
    // Write must return a non-nil error if it returns n < len(p).  
    // Write must not modify the slice data, even temporarily.  
    // Implementations must not retain p.  
    Write(p []byte) (n int, err error)  
}
```

```
type ByteCounter int

func (c *ByteCounter) Write(p []byte) (int, error) {
    *c += ByteCounter(len(p)) // convert int to ByteCounter
    return len(p), nil
}

var c ByteCounter
c.Write([]byte("hello"))
fmt.Println(c) // "5", = len("hello")
```

```
package fmt

// The String method is used to print values passed
// as an operand to any format that accepts a string
// or to an unformatted printer such as Print.
type Stringer interface {
    String() string
}
```

Композиция интерфейсов

```
package io

type Reader interface {
    Read(p []byte) (n int, err error)
}

type Closer interface {
    Close() error
}

type ReadWriter interface {
    Reader
    Writer
}

type ReadWriteCloser interface {
    Reader
    Writer
    Closer
}
```

Встраивание интерфейсов в struct

```
package sort

type Interface interface {
    Len() int
    Less(i, j int) bool
    Swap(i, j int)
}

func Reverse(data Interface) Interface {
    return &reverse{data}
}

type reverse struct {
    Interface
}

func (r reverse) Less(i, j int) bool {
    return r.Interface.Less(j, i)
}
```

Соответствие интерфейсу

```
var w io.Writer
w = os.Stdout           // OK: *os.File has Write method
w = new(bytes.Buffer)   // OK: *bytes.Buffer has Write method
w = time.Second         // compile error: time.Duration lacks Write method
```


Тип any

```
type any = interface{}  
  
// any - это alias для interface{}  
var v any  
v = true  
v = 12.34  
v = "hello"  
v = map[string]int{"one": 1}  
v = new(bytes.Buffer)
```

Проверка соответствия интерфейсу

```
// *bytes.Buffer must satisfy io.Writer  
var _ io.Writer = (*bytes.Buffer)(nil)
```

Значения интерфейсов

```
var w io.Writer
w = os.Stdout
w = new(bytes.Buffer)
w = nil

var x any = time.Now()
```

Ловушка с nil указателем

```
var buf *bytes.Buffer
if debug {
    buf = new(bytes.Buffer) // enable collection of output
}
f(buf)

// If out is non-nil, output will be written to it.
func f(out io.Writer) {
    // ...do something...
    if out != nil {
        out.Write([]byte("done!\n"))
    }
}
```

```
var buf io.Writer
if debug {
    buf = new(bytes.Buffer) // enable collection of output
}
f(buf) // OK
```

Type Assertions (конкретный тип)

```
var w io.Writer
w = os.Stdout
f := w.(*os.File)           // success: f == os.Stdout
c := w.(*bytes.Buffer)      // panic: interface holds *os.File, not
↪ *bytes.Buffer
```

Интерфейсный тип:

```
var w io.Writer
w = os.Stdout
rw := w.(io.ReadWriter)     // success: *os.File has both Read and Write
w = new(ByteCounter)
rw = w.(io.ReadWriter)      // panic: *ByteCounter has no Read method
```

Type Assertions (без паники)

```
var w io.Writer
var rw io.ReadWriter

w = rw           // io.ReadWriter is assignable to io.Writer
w = rw.(io.Writer) // fails only if rw == nil
```

С проверкой:

```
var w io.Writer = os.Stdout
f, ok := w.(*os.File)           // success: ok, f == os.Stdout
b, ok := w.(*bytes.Buffer)      // failure: !ok, b == nil
```

Type Switches (длинный путь)

```
func sqlQuote(x any) string {  
    if x == nil {  
        return "NULL"  
    } else if _, ok := x.(int); ok {  
        return fmt.Sprintf("%d", x)  
    } else if _, ok := x.(uint); ok {  
        return fmt.Sprintf("%d", x)  
    } else if b, ok := x.(bool); ok {  
        if b { return "TRUE" }  
        return "FALSE"  
    } else if s, ok := x.(string); ok {  
        return sqlQuoteString(s)  
    } else {  
        panic(fmt.Sprintf("unexpected type %T: %v", x, x))  
    }  
}
```


Type Switches (синтаксис)

```
switch x.(type) {  
case nil:  
    // ...  
case int, uint:  
    // ...  
case bool:  
    // ...  
case string:  
    // ...  
default:  
    // ...  
}
```

```
switch x := x.(type) { /* ... */ }
```

Type Switches (пример)

```
func sqlQuote(x any) string {  
    switch v := x.(type) {  
    case nil:  
        return "NULL"  
    case int, uint:  
        return fmt.Sprintf("%d", v) // v has type any here.  
    case bool:  
        if v { return "TRUE" }  
        return "FALSE"  
    case string:  
        return sqlQuoteString(v)  
    default:  
        panic(fmt.Sprintf("unexpected type %T: %v", v, v))  
    }  
}
```

```
switch s := suit(drawCard()); s {  
case "Spades":  
case "Hearts":  
case "Diamonds":  
case "Clubs":  
default:  
    panic(fmt.Sprintf("invalid suit %q", s)) // Joker?  
}  
  
func Reset(x *Buffer) {  
    if x == nil {  
        panic("x is nil") // unnecessary!  
    }  
    x.elements = nil  
}
```

defer (без defer)

```
func CopyFile(dstName, srcName string) (written int64, err error) {  
    src, err := os.Open(srcName)  
    if err != nil { return }  
    dst, err := os.Create(dstName)  
    if err != nil { return }  
    written, err = io.Copy(dst, src)  
    dst.Close()  
    src.Close()  
    return  
}
```

defer (c defer)

```
func CopyFile(dstName, srcName string) (written int64, err error) {  
    src, err := os.Open(srcName)  
    if err != nil { return }  
    defer src.Close()  
  
    dst, err := os.Create(dstName)  
    if err != nil { return }  
    defer dst.Close()  
  
    return io.Copy(dst, src)  
}
```

defer (порядок выполнения)

```
func a() {  
    i := 0  
    defer fmt.Println(i)  
    i++  
    return  
}  
  
a() // Output: 0
```

defer (стек)

```
func b() {  
    for i := 0; i < 4; i++ {  
        defer fmt.Println(i)  
    }  
}  
  
fmt.Println(b())  
// Output: 3 2 1 0
```

defer (замыкание)

```
func c() (i int) {  
    defer func() { i++ }()  
    return 1  
}  
  
fmt.Println(c()) // Output: 2
```


defer (цепочка вызовов)

```
func main() {  
    f(3)  
}  
  
func f(x int) {  
    fmt.Printf("%d\n", x+0/x) // panics if x == 0  
    defer fmt.Printf("defer %d\n", x)  
    f(x - 1)  
}  
  
// Output:  
// f(3)  
// f(2)  
// f(1)  
// defer 1  
// defer 2  
// defer 3
```

```
func Parse(input string) (s *Syntax, err error) {  
    defer func() {  
        if p := recover(); p != nil {  
            err = fmt.Errorf("internal error: %v", p)  
        }  
    }()  
    // ...parser...  
}
```

recover (сложный пример)

```
func main() {  
    defer func() {  
        recover()  
        fmt.Println("Checkpoint 1")  
        panic(1)  
    }()  
    defer func() {  
        recover()  
        fmt.Println("Checkpoint 2")  
        panic(2)  
    }()  
    panic(999)  
}
```

```
// Output:  
// Checkpoint 2  
// Checkpoint 1  
// panic: 999 [recovered]  
//      panic: 2 [recovered]  
//      panic: 1
```

```
type error interface {
    Error() string
}

type Unwrapper interface {
    Unwrap() error
}

// Unwrap
errors.Unwrap(fmt.Errorf("... %w ...", ..., err, ...)) == err
```

Is:

```
if errors.Is(err, os.ErrExist)

// вместо
if err == os.ErrExist
```

As:

```
var perr *os.PathError
if errors.As(err, &perr) {
    fmt.Println(perr.Path)
}

// вместо
if perr, ok := err.(*os.PathError); ok {
    fmt.Println(perr.Path)
}
```

Дженерики

Дженерики позволяют писать функции и типы, которые работают с любыми типами, сохраняя типобезопасность.

```
func SumInts(m map[string]int64) int64 {  
    var s int64  
    for _, v := range m { s += v }  
    return s  
}  
  
func SumFloats(m map[string]float64) float64 {  
    var s float64  
    for _, v := range m { s += v }  
    return s  
}
```

```
func Sum[K comparable, V int64 | float64](m map[K]V) V {  
    var s V  
    for _, v := range m { s += v }  
    return s  
}
```

Constraints (Ограничения)

Constraints — интерфейсы, определяющие, какие типы могут быть использованы с дженериком.

```
// Встроенный пакет constraints
type Signed interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64
}

type Unsigned interface {
    ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 | ~uintptr
}

type Integer interface {
    Signed | Unsigned
}

type Float interface {
    ~float32 | ~float64
```

Уравнения на дженериках

```
// slices.Sort: func Sort[S ~[]E, E cmp.Ordered](x S)

type List []int
var list List
slices.Sort(list)
```

Уравнения, которые решает компилятор:

```
S := List                                // List должен быть присваиваем S
under(S) == []E                          // Базовый тип S - []E
E in cmp.Ordered                         // E удовлетворяет ограничению

S -> List
E -> int (т.к. under(List) = []int, значит []int == []E)
```


Унификация (Unification)

```
func equal[P comparable](x, y P) bool { return x == y }  
  
var strEq func(string, string) bool = equal
```

- Если тип параметр P уже имеет выведенный тип — используется он
- Если нет и сравнивается с типом T — P устанавливается в T
- Если два параметра без типа унифицируются — они связываются

Дженерики над базовыми типами и дизъюнкция

```
type MyInt int

// Без ~ - только int
func WithoutTilde[T int](x T) T { return x }
// WithoutTilde(MyInt(5)) // Ошибка

// С ~ - любой тип с базовым типом int
func WithTilde[T ~int](x T) T { return x }
WithTilde(MyInt(5)) // OK

func Min[T ~int | ~float64](a, b T) T {
    if a < b { return a }
    return b
}
```

Дженерик any

```
// Это одно и то же:
type any = interface{}

func Print[T any](v T) {
    fmt.Println(v)
}

func Print2[T interface{}](v T) {
    fmt.Println(v)
}
```

any как ограничение означает "абсолютно любой тип"

```
// Можно передать что угодно
Print(42)
Print("hello")
Print(struct{ name string }{"alice"})
Print(map[string]int{"a": 1})
```

Псевдо-дженерики vs Настоящие дженерики

```
// "Дженерик" через интерфейс
type Stringer interface {
    String() string
}

func PrintString(s Stringer) {
    fmt.Println(s.String())
}

// Работает только с типами, имеющими метод String()
```

```
func PrintAny[T any](v T) {
    fmt.Println(v)
}
```

- Интерфейсы — *структурная типизация* (что тип имеет)
- Дженерики — *параметрический полиморфизм* (каким типом является)

Вывод типов в рекурсивных функциях

```
func fact[P ~int | ~float64](n P) P {  
    if n <= 1 { return 1 }  
    return fact(n-1) * n  
}
```

$P := P$ — нет новой информации

// Компилятор временно переименовывает для вывода типов

```
func fact[P ~int | ~float64](n P) P {  
    if n <= 1 { return 1 }  
    return helper(n-1) * n  
}
```

```
func helper[Q ~int | ~float64](n Q) Q {  
    return fact(n)  
}
```

$Q := P \rightarrow Q \rightarrow P$

Базовые constraints

```
func Find[T comparable](slice []T, val T) int {  
    for i, v := range slice {  
        if v == val { return i }  
    }  
    return -1  
}
```

```
func Max[T cmp.Ordered](a, b T) T {  
    if a > b { return a }  
    return b  
}
```

```
func Double[T interface{ ~int | ~float64 }](x T) T {  
    return x * 2  
}
```

```
func Print[T any](v T) { fmt.Println(v) }
```